

EcoVision

Waste Detection and Cleanup Assistant

Дарья Чебуркова Илья Блинов Всеволод Ершов
Константин Румовский Эльдар Имангалиев

Формат проекта	End-to-end CV-система с пользовательским интерфейсом
Задача	Object detection мусора на изображениях и формирование краткого отчета
Датасет	TACO: Trash Annotations in Context, преобразованный в YOLO-формат
Репозиторий	EcoVision

Содержание

1	Введение	3
1.1	Описание проблемы	3
1.2	Потенциальные пользователи и сценарии	3
1.3	Постановка задачи	3
1.4	Этапы проекта	3
2	Обзор источников	4
3	Подготовка данных	5
3.1	Укрупнение классов	6
3.2	Датасет с одним классом	7
4	Архитектура модели	7
5	Обучение	8
5.1	Структура реализации	8
5.2	Гиперпараметры	8
6	Результаты	9
7	Веб-приложение	10
8	Выводы и дальнейшее развитие	11
8.1	Ключевые результаты	11
8.2	Оставшиеся ограничения	11
8.3	Направления улучшения	12
9	Распределение задач в команде	12

1 Введение

1.1 Описание проблемы

Экологический мониторинг часто требует быстрой оценки загрязненности территории. Ручной осмотр фотографий занимает время, плохо масштабируется и зависит от субъективности наблюдателя. EcoVision решает прикладную задачу: по одному изображению автоматически найти видимые объекты мусора и дать понятный пользователю отчет.

1.2 Потенциальные пользователи и сценарии

- муниципальные службы и волонтерские команды, которым нужно быстро оценивать точки загрязнения;
- экологические инициативы, собирающие фотоотчеты с территорий;
- команды экологического мониторинга, которым требуется воспроизводимый CV-пайплайн от данных до интерфейса;
- прототипы мобильных или web-сервисов для сортировки и уборки отходов.

1.3 Постановка задачи

Таблица 1: Постановка задачи и инструменты

Параметр	Описание
Тип задачи	Детекция: bounding boxes, class labels, confidence scores
Данные	TACO <code>annotations.json</code> , COCO-like annotations, изображения реального мусора
Модель	Ultralytics YOLO; финальный эксперимент на one-label классе <code>waste</code>
Инструменты	Python, PyTorch/Ultralytics, PIL, pandas, Streamlit, YAML, Jupyter
Интерфейс	Веб-приложение с загрузкой изображения и визуальным отчетом

1.4 Этапы проекта

1. Изучение TACO, анализ категорий.
2. Конвертация COCO-like аннотаций в YOLO-format labels.
3. Разделение данных на train/validation и подготовка `dataset.yaml`.
4. Обучение YOLO-моделей и выбор лучшей.
5. Оценка метрик и анализ ошибок.
6. Реализация пайплайна инференса.

2 Обзор источников

Для задачи детекции в прикладных CV-системах чаще всего используют несколько групп подходов. Two-stage detectors, например Faster R-CNN, строят region proposals и затем уточняют классы и координаты объектов; такой подход исторически дает сильное качество на benchmark-датасетах, но требует более сложной настройки инференса и обычно дороже по задержке [1]. Mask R-CNN расширяет Faster R-CNN веткой instance segmentation и полезен, когда нужна точная маска каждого объекта, однако для EcoVision исходные данные и целевой интерфейс ориентированы на bounding boxes, а не на pixel-level masks [2]. Детекторы на основе трансформеров, такие как DETR, формулируют задачу детекции как предсказание множества объектов и уменьшают объём вручную разработанной постобработки, однако могут быть более требовательны к объёму данных, времени обучения и вычислительным ресурсам [3].

YOLO-семейство относится к one-stage detectors: модель предсказывает bounding boxes и вероятности классов за один прямой проход [4]. Пользовательский сценарий предполагает интерактивную загрузку изображения, короткое время ожидания, простую интеграцию с web-интерфейсом и возможность дальнейшего развертывания на недорогом GPU или CPU-backed сервисе. Современные версии YOLO обеспечивают хороший баланс между скоростью и точностью. YOLOv4 показывает, что улучшения архитектуры и процесса обучения позволяют сохранить высокое качество детекции при работе real-time [5], а YOLOv7 демонстрирует конкурентный баланс между скоростью и точностью [6].

Таблица 2: Сравнение подходов к решению задачи

Подход	Сильные стороны	Ограничения для EcoVision
Faster R-CNN [1]	Сильный бейзлайн для детекции; region proposal network повышает качество локализации по сравнению с более ранними R-CNN-подходами	Более тяжелая двухстадийная схема увеличивает задержку и усложняет инференс для интерактивного сервиса
Mask R-CNN [2]	Добавляет instance masks и может оценивать форму/площадь загрязнения	Требует сегментационной разметки; в текущем датасете используется bbox-разметка, поэтому маски пришлось бы получать дополнительно
DETR [3]	Мало вручную настраиваемых параметров	Более дорогой training и потенциально большая потребность в данных; для небольшого доменного датасета это повышает риск нестабильного качества
YOLO [4, 5, 6]	Одностадийный инференс, низкая задержка, хороший баланс скорости и качества для real-time задач	Требует аккуратной работы с мелкими объектами, балансировки классов и подбор порога

Итоговый выбор YOLO основан на эксплуатационных требованиях EcoVision. Система должна быстро отвечать на пользовательскую загрузку изображения, возвращать координаты объектов в формате, удобном для отчета, и оставаться достаточно простой для дальнейшего развертывания как веб-сервис. При этом YOLO возможность масштабировать в дальнейшем, дообучать на новых изображениях, настраивать пороги для детекции, добавлять аугментации для мелких объектов.

3 Подготовка данных

В качестве исходного источника используется TACO (Trash Annotations in Context). В проекте аннотации читаются из `data/raw/annotations.json`, категории анализируются в notebook `dataset_preparation.ipynb`, а финальная конвертация выполняется скриптами `scripts/prepare_dataset.py` и `scripts/prepare_one_label_dataset.py`.

Таблица 3: Статистика подготовленного YOLO-датасета

Параметр	Значение
Кол-во изображений на обучении	1200
Кол-во изображений на тестировании	300
Всего изображений	1500
Всего объектов	4784
YOLO label format	class_id x_center y_center width height, normalized

3.1 Укрупнение классов

Исходные категории TACO разрежены и дисбалансны, поэтому они были объединены в шесть категорий: `rigid_plastic`, `soft_plastic`, `paper/cardboard`, `metal`, `glass`, `other`. Такое укрупнение снижает шум разметки и делает результат более понятным для пользователя.

Таблица 4: Распределение объектов

class_id	class_name	objects
0	rigid_plastic	1395
1	soft_plastic	857
2	paper/cardboard	497
3	metal	550
4	glass	254
5	other	1231

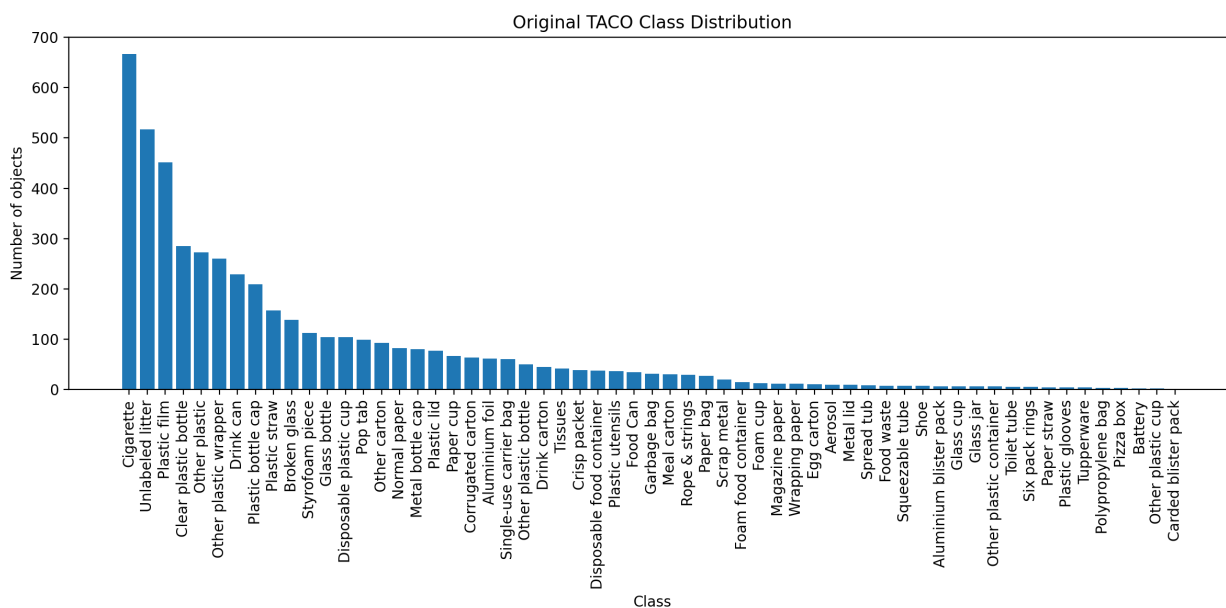


Рис. 1: Распределение исходных TACO-категорий: заметен длинный хвост редких классов.

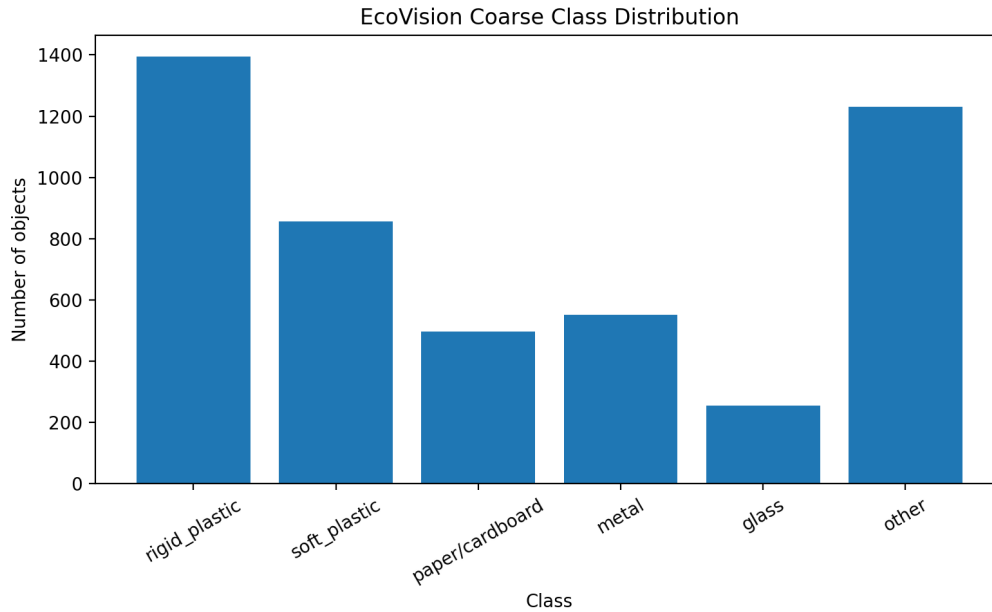


Рис. 2: Распределение после remapping в шесть coarse classes.

3.2 Датасет с одним классом

Дополнительно подготовлен одноклассовый вариант: весь мусор относится к одному классу waste. Этот вариант был подготовлен для проведения сравнения обучения и оценки качества многоклассовых и одноклассовых моделей.

Таблица 5: Статистика one-label датасета

split	images	objects
Обучение	1200	3572
Тестирование	300	1212
Всего	1500	4784

4 Архитектура модели

Проект построен вокруг Ultralytics YOLO. Модель получает изображение, возвращает список boxes, class IDs и confidence scores. Сырые предсказания затем приводятся к единому формату проекта.

Для обучения были выбраны архитектуры YOLOv8s и YOLO11s. В процессе разработки эти модели были обучены на двух типах задач: one-label (детекция всего мусора как класс waste), multi-label (классы, описанные в предыдущем разделе). Далее в тексте работы будет приведено сравнение четырех моделей, из которых были выбраны две лучших: одна для one-label, другая – для multi-label. В итоговой реализации пользователь сам выбирает, какую модель использовать.

5 Обучение

5.1 Структура реализации

Таблица 6: Основные компоненты реализации

Компонент	Назначение
Подготовка датасета	COCO-like TACO to YOLO labels, class remapping, split, statistics
Обучение	Запуск пайплайна YOLOv8s/YOLO11s, сохранение лучших весов
Оценка качества	Метрики, графики, error analysis
Инференс	Реализация загрузки модели, запуска инференса
Отчет по загрязнению	Вывод статистики и рекомендации
Веб-интерфейс	Streamlit, интерфейс настройки модели, загрузка изображений, отчет по изображению с визуализацией

5.2 Гиперпараметры

Таблица 7: Гиперпараметры

Параметр	Значение
Epochs	50
Image size	640
Batch	8
Workers	6
Seed	42
Confidence threshold	0.25
IoU threshold	0.50
Optimizer/loss	Ultralytics YOLO defaults: box loss, class loss, DFL loss tracked through training logs
Augmentations	YOLO default training augmentations; explicit resize through <code>imgsz=640</code>

6 Результаты

Метрики рассчитаны на выборке для тестирования.

Таблица 8: Сравнение результатов обученных моделей

Постановка	Модель	Precision	Recall	mAP@50	mAP@50– 95	Время обуче- ния, мин
Многоклассовая	YOLOv8s	0.2669	0.2141	0.1696	0.1251	33.14
Многоклассовая	YOLO11s	0.2663	0.2262	0.1753	0.1203	25.83
Один класс	YOLOv8s	0.5679	0.3853	0.4061	0.2703	25.67
Один класс	YOLO11s	0.5576	0.3663	0.3893	0.2646	27.26

Для веб-приложения была выбрана одноклассовая и многоклассовая YOLOv8s.

Таблица 9: Error analysis для лучшей одноклассовой модели

Error type	Count
Всего изображений	80
Всего объектов	323
False positive	106
Пропущенные объекты	186
Пропущенные небольшие объекты	135

Основная проблема – пропуск мелких объектов. Это ожидаемо для TACO: мусор часто занимает малую площадь кадра, частично закрыт фоном, листьями или тенями, а сами классы несбалансированы.

Ниже представлены ошибки, которые может допускать модель.



Рис. 3: False positive и false negative



Рис. 4: Пропущенные мелкие объекты

7 Веб-приложение

Пользовательский интерфейс реализован на Streamlit. Пользователь может выбрать путь к модели, порог для детекции, затем загрузить одно изображение. Приложение показывает исходный кадр, кадр с предсказаниями с bounding boxes, итоговое саммари в JSON. Демонстрация работы представлена в репозитории в папке results.



Рис. 5: Контрольный пример inference: найдено 4 объекта **waste**, pollution level = **medium**.

8 Выводы и дальнейшее развитие

8.1 Ключевые результаты

- Собран полный пайплайн: подготовка данных, обучение, оценка, инференс и Streamlit-интерфейс.
- ТАСО преобразован в YOLO-format: 1500 изображений, 4784 объекта, train/validation split 1200/300.
- Реализованы два варианта постановки: многоклассовый и одноклассовый.
- Две финальные модели подключены к приложению.
- Приложение возвращает boxes, пользовательский отчет: таблицу, статистику, уровень загрязнения и рекомендации.

8.2 Оставшиеся ограничения

- Recall пока на невысоком уровне, модель склонна пропускать мелкий мусор.
- Streamlit-интерфейс работает как локальный прототип, а не как масштабируемый сервис.

8.3 Направления улучшения

- добавить больше данных и аугментаций для мелких объектов;
- сравнить обученные модели с более крупными YOLO-моделями, RT-DETR и Faster R-CNN;
- добавить экспорт отчета в PDF/CSV и API endpoint для интеграции с мобильным приложением.

9 Распределение задач в команде

Таблица 10: Team task distribution

Участник	Зона ответственности	Основные артефакты
Дарья Чебуркова	Dataset Preparation & Class Remapping	<code>data/raw/</code> , class mapping, dataset statistics, class distribution plots
Илья Блинов	Annotation Conversion & Training Setup	<code>configs/dataset.yaml</code> , YOLO labels, train/val split, training config
Эльдар Имангалиев	Model Training, Evaluation & Error Analysis	<code>models/best.pt</code> , metrics summary, error analysis, evaluation artifacts
Константин Румовский	Inference Pipeline, Visualization & Report Logic	<code>src/inference.py</code> , <code>src/visualization.py</code> , <code>src/report.py</code> , structured detections
Ершов Всеволод	Streamlit App, Repository, Report & Presentation	<code>app.py</code> , README, usage scenario, final report and slides

Список литературы

- [1] Shaoqing Ren, Kaiming He, Ross Girshick, Jian Sun. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. arXiv:1506.01497, 2015. <https://arxiv.org/abs/1506.01497>
- [2] Kaiming He, Georgia Gkioxari, Piotr Dollár, Ross Girshick. *Mask R-CNN*. arXiv:1703.06870, 2017. <https://arxiv.org/abs/1703.06870>
- [3] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, Sergey Zagoruyko. *End-to-End Object Detection with Transformers*. arXiv:2005.12872, 2020. <https://arxiv.org/abs/2005.12872>
- [4] Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi. *You Only Look Once: Unified, Real-Time Object Detection*. arXiv:1506.02640, 2015. <https://arxiv.org/abs/1506.02640>
- [5] Alexey Bochkovskiy, Chien-Yao Wang, Hong-Yuan Mark Liao. *YOLOv4: Optimal Speed and Accuracy of Object Detection*. arXiv:2004.10934, 2020. <https://arxiv.org/abs/2004.10934>
- [6] Chien-Yao Wang, Alexey Bochkovskiy, Hong-Yuan Mark Liao. *YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors*. arXiv:2207.02696, 2022. <https://arxiv.org/abs/2207.02696>
- [7] Juan Terven, Diana Cordova-Esparza. *A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS*. arXiv:2304.00501, 2023. <https://arxiv.org/abs/2304.00501>